

# OwenHuangAssignment03

March 31, 2026

## 0.1 Exercise 01

Write code to read data from: [https://data.giss.nasa.gov/gistemp/taledata\\_v3/GLB.Ts+dSST.txt](https://data.giss.nasa.gov/gistemp/taledata_v3/GLB.Ts+dSST.txt)  
 . (You can use data from the file `GLB.Ts+dSST.csv` instead)

Keep all monthly temperatures (from Jan to Dec) between 1980 and 2015 ONLY (ignore any other data), then:

- Find minimum temperature in Jan between 1980 - 2015
- Find average temperature in each year between 1980 - 2015
- Find temperature in Jan 2000
- Find temperature in Jun 2009

```
[81]: # Find minimum temperature in Jan between 1980 - 2015
import pandas as pd

monthly_columns = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug",
                  ↪ "Sep", "Oct", "Nov", "Dec"]

nasa = pd.read_csv("GLB.Ts+dSST.csv", skiprows=1)
nasa[monthly_columns] = nasa[monthly_columns].apply(pd.to_numeric,
                  ↪ errors="coerce")

temps_1980to2015 = nasa.loc[nasa["Year"].between(1980, 2015), ["Year",
                  ↪ *monthly_columns]].reset_index(drop=True)

min_temp = temps_1980to2015["Jan"].min()
print(float(min_temp))
```

0.1

```
[82]: # Find average temperature in each year between 1980 - 2015
average_temperature_yearly = temps_1980to2015.copy()
average_temperature_yearly["Average"] =
    ↪ average_temperature_yearly[monthly_columns].mean(axis=1)

average_temperature_yearly[["Year", "Average"]]
```

```
[82]:
```

|    | Year | Average  |
|----|------|----------|
| 0  | 1980 | 0.270833 |
| 1  | 1981 | 0.330833 |
| 2  | 1982 | 0.132500 |
| 3  | 1983 | 0.315000 |
| 4  | 1984 | 0.160833 |
| 5  | 1985 | 0.118333 |
| 6  | 1986 | 0.180000 |
| 7  | 1987 | 0.332500 |
| 8  | 1988 | 0.407500 |
| 9  | 1989 | 0.289167 |
| 10 | 1990 | 0.440833 |
| 11 | 1991 | 0.414167 |
| 12 | 1992 | 0.223333 |
| 13 | 1993 | 0.238333 |
| 14 | 1994 | 0.307500 |
| 15 | 1995 | 0.445000 |
| 16 | 1996 | 0.335000 |
| 17 | 1997 | 0.468333 |
| 18 | 1998 | 0.622500 |
| 19 | 1999 | 0.396667 |
| 20 | 2000 | 0.399167 |
| 21 | 2001 | 0.529167 |
| 22 | 2002 | 0.620833 |
| 23 | 2003 | 0.605000 |
| 24 | 2004 | 0.531667 |
| 25 | 2005 | 0.666667 |
| 26 | 2006 | 0.616667 |
| 27 | 2007 | 0.637500 |
| 28 | 2008 | 0.515833 |
| 29 | 2009 | 0.631667 |
| 30 | 2010 | 0.696667 |
| 31 | 2011 | 0.582500 |
| 32 | 2012 | 0.617500 |
| 33 | 2013 | 0.646667 |
| 34 | 2014 | 0.731667 |
| 35 | 2015 | 0.865833 |

```
[83]: # Find temperature in Jan 2000
Jan2000Temp = float(temps_1980to2015.loc[temps_1980to2015["Year"] == 2000,
↪ "Jan"].iloc[0])
Jan2000Temp
```

```
[83]: 0.23
```

```
[84]: # Find temperature in Jun 2009
```

```
Jun2009Temp = float(temps_1980to2015.loc[temps_1980to2015["Year"] == 2009,
↪ "Jun"].iloc[0])
Jun2009Temp
```

[84]: 0.65

## 0.2 Extra Credit (Optional)

answers.txt contains students answers of exam questions (each line contains student\_id, then answers made by that student)

- The first number is the student's ID number. The following 25 letters are the student responses to the exam. If there is no letter following a comma, this means the student skipped answering the question.
- Write code to read the data from the file, compare students answers to answers key (1 point for each correct answer), then find:
  - The grade for each student.
  - Id of student of the highest score.
  - Id of student of the lowest score.

```
answerkey = "B,A,D,D,C,B,D,A,C,C,D,B,A,B,A,C,B,D,A,C,A,A,B,D,D"
```

[85]: answerkey = "B,A,D,D,C,B,D,A,C,C,D,B,A,B,A,C,B,D,A,C,A,A,B,D,D"

[86]: *# The grade for each student*

```
correct_answers = answerkey.split(",")
student_answers = pd.read_csv("answers.txt", header=None, keep_default_na=False)

student_answers.columns = ["student_id", *[f"Q{i}" for i in range(1, 26)]]
responses = student_answers.iloc[:, 1:]
grades = student_answers[["student_id"]].copy()
grades["score"] = responses.eq(correct_answers).sum(axis=1)

grades
```

[86]:

|    | student_id | score |
|----|------------|-------|
| 0  | N00000001  | 16    |
| 1  | N00000002  | 18    |
| 2  | N00000003  | 21    |
| 3  | N00000004  | 19    |
| 4  | N00000005  | 21    |
| 5  | N00000006  | 18    |
| 6  | N00000007  | 22    |
| 7  | N00000008  | 18    |
| 8  | N00000009  | 22    |
| 9  | N00000010  | 19    |
| 10 | N00000011  | 22    |
| 11 | N00000012  | 19    |

|    |           |    |
|----|-----------|----|
| 12 | N00000013 | 19 |
| 13 | N00000014 | 20 |
| 14 | N00000015 | 19 |
| 15 | N00000016 | 23 |
| 16 | N00000017 | 18 |
| 17 | N00000018 | 20 |
| 18 | N00000019 | 20 |
| 19 | N00000020 | 18 |

```
[87]: # Id of student of the highest score
      id_highest = grades.loc[grades["score"].idxmax(), "student_id"]
      print(id_highest)
```

N00000016

```
[88]: # Id of student of the lowest score
      id_lowest = grades.loc[grades["score"].idxmin(), "student_id"]
      print(id_lowest)
```

N00000001